



Solution Architecture

TASCO Insurance × VETC Growth Platform — Architecture

iorta TechNXT

Version 1.0 · 07 October 2026

Prepared by iorta TechNXT for TASCO Insurance. Confidential: for the use of TASCO Insurance and VETC only.

Contents

1. Business context in one paragraph
2. Architecture principles
3. C4 Level 1 — System context
4. C4 Level 2 — Containers
5. C4 Level 3 — Components (API server)
6. Hexagonal layering and the dependency rule
7. Key flows
8. Event-driven design
9. Configurability model
10. Scalability to 6 million vehicles
11. Quality attributes (targets)
12. Evolution roadmap

Document	Version	Date	Owner	Status
Solution Architecture	1.0	07 October 2026	iorta TechNXT Architecture	Issued for submission

Scope: the TASCO Growth Platform codebase (`/src` , `/config` , `/db`). Companion documents: [Integration Architecture](#), [Deployment and Infrastructure Architecture](#), [Data Architecture](#), [Security Architecture](#), [AI and Prompt Governance](#) and the architecture decision records listed in the [Architecture Documentation index](#).

1. Business context in one paragraph

VETC runs electronic toll collection for about 6 million vehicles. Its records hold a plate, a phone number and engagement signals for every vehicle. Only about 1 in 10 records has a verified motor insurance certificate (`src/adapters/integrations/syntheticVetSource.js` reproduces this). TASCO Insurance wants to:

- win **new business**, meaning uninsured, lapsed and newly registered vehicles;
- **retain** its own TNDS (compulsory third-party liability) customers;
- work **with** the partners (banks, showrooms, agents, fleets, inspection centres) who close most TNDS sales today, not against them;
- give customers a trustworthy digital experience, including claims FNOL.

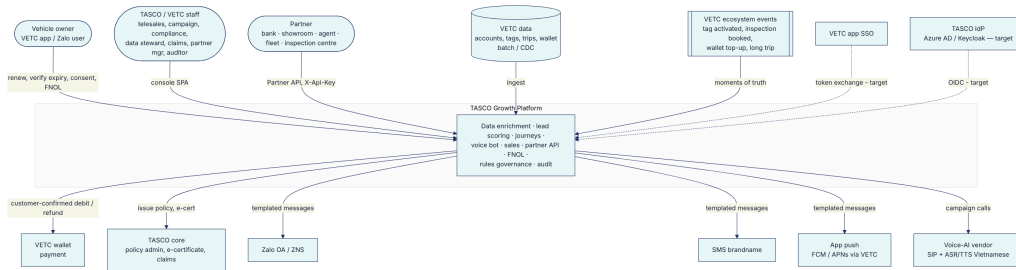
The TNDS premium is set by regulation (`config/rules/tariff.tnds_car.json` , Decree 67/2023/ND-CP; **confirm with TASCO legal**). Price therefore cannot be the lever. The platform competes on four things instead: **data, timing, channel and service value**.

2. Architecture principles

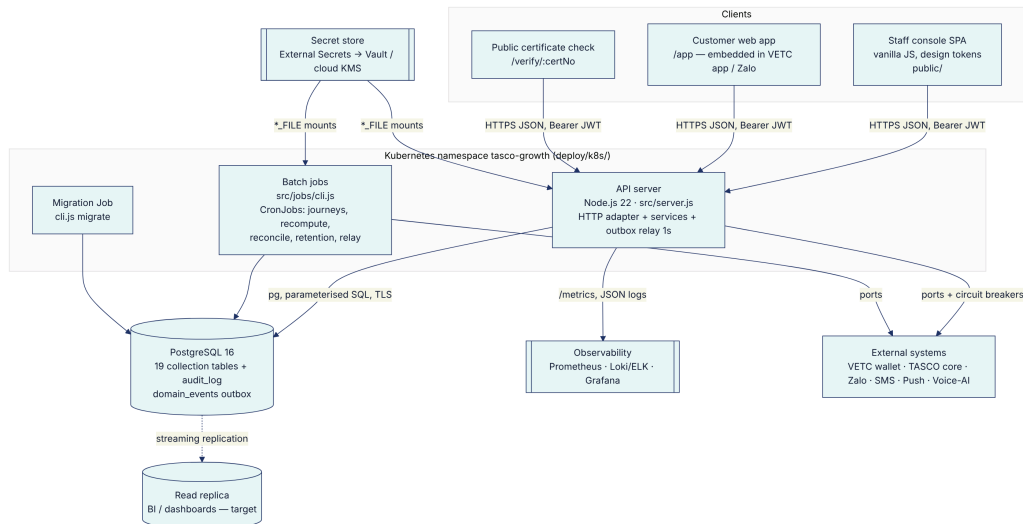
#	Principle	How the code enforces it
P1	Domain at the centre, adapters at the edge (hexagonal)	<code>src/domain/*</code> imports only <code>src/shared/*</code> and <code>src/rules/*</code> . Adapters are chosen only in <code>src/bootstrap/container.js</code> . See ADR-001 .
P2	Business rules are data, not code	21 rule kinds in <code>config/rules/*.json</code> , evaluated by a safe JSON Logic subset (<code>src/rules/jsonLogic.js</code>). Changes go through maker-checker (<code>src/application/rulesService.js</code>). See ADR-003 .
P3	Secure by default, deny by default	Every route declares <code>auth/perm/schemas</code> (<code>src/adapters/http/routes.js</code>). Validation rejects unknown fields (<code>src/shared/validation.js</code>). PII is encrypted per field (<code>codec.js</code>). Production refuses to boot without secrets or a database, or with demo mode on (<code>src/shared/config.js</code>).
P4	Explainability over black boxes	The lead score is a weighted sum of named factors with reason strings (<code>src/domain/leads.js#score</code>). Each NBA result carries a <code>ruleId</code> . Golden-record fields carry lineage and confidence (<code>src/domain/enrichment.js</code>).
P5	Consent and trust first	Every outbound touch passes <code>canContact()</code> (<code>src/domain/contactPolicy.js</code>) and the copy guard. The voice bot discloses that it is automated and verifies the plate first (<code>src/domain/voicebot.js</code>).
P6	Idempotent, at-least-once integration	Orders use idempotency keys, and the quote is claimed atomically before payment (<code>salesService.purchase</code>). Outbound calls are wrapped in <code>timeout + retry + circuit breaker</code> (<code>src/shared/resilience.js</code>). Event handlers are named and tracked per event (<code>outboxEventBus.js</code>).
P7	Minimal supply chain	One runtime dependency (<code>pg</code>). Crypto, HTTP, routing, metrics and validation come from <code>node:*</code> built-ins. See ADR-002 .
P8	Twelve-factor, stateless processes	Config comes from <code>env</code> or <code>*_FILE</code> . Logs are JSON on <code>stdout</code> . State lives in Postgres. Shutdown is graceful (<code>src/server.js</code>).
P9	Same behaviour in every environment	The in-memory and Postgres stores share <code>codec.js</code> , <code>query.js</code> and <code>auditChain.js</code> , so encryption, filtering and audit chaining behave the same (ADR-004).
P10	Tamper-evident accountability	A hash-chained <code>audit_log</code> with database triggers that block UPDATE, DELETE (<code>db/migrations/001_init.sql</code>) and TRUNCATE (<code>002_integrity_hardening.sql</code>) (ADR-008).

3. C4 Level 1 — System context

The platform sits between VETC's customer base and channels, TASCO's core systems and the partner network.



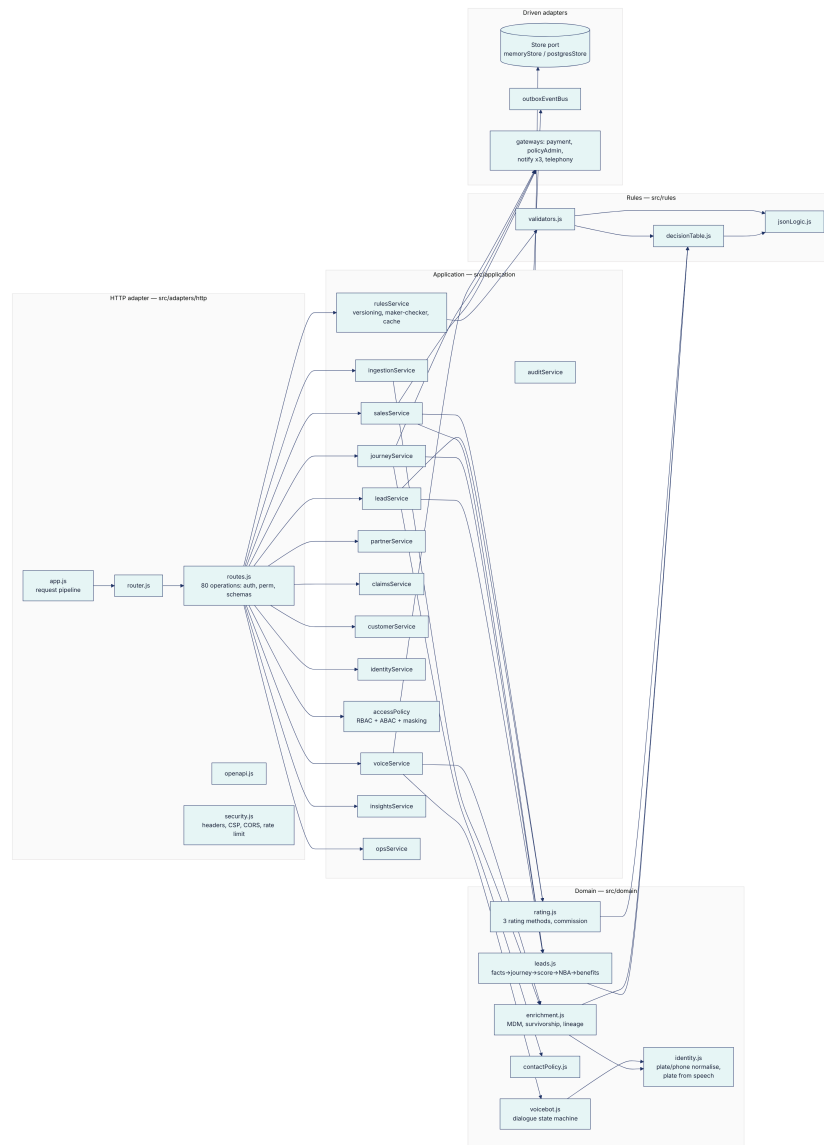
4. C4 Level 2 — Containers



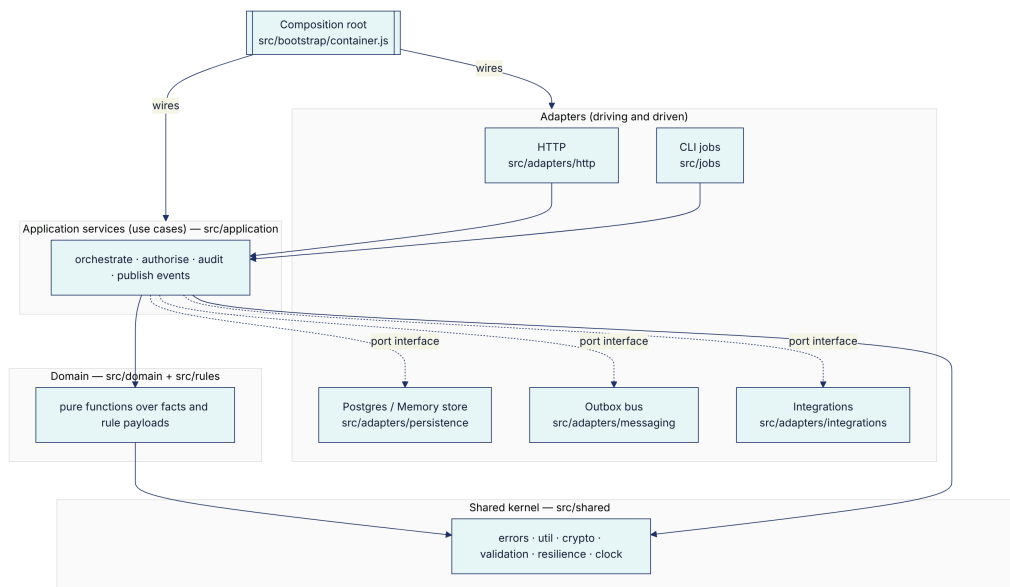
Container	Technology	Responsibility	Scaling
API server	Node.js 22, node:http, src/server.js	All synchronous APIs, static SPA, /health/*, /metrics, /api/openapi.json; background outbox relay every 1 s	Horizontal (stateless). HPA on CPU. The outbox relay is safe on N replicas (FOR UPDATE SKIP LOCKED).
Batch jobs	Same image, node src/jobs/cli.js <job>	journeys, recompute, reconcile, retention, relay, migrate, seed (requires DEMO_MODE), rules, openapi	One pod per CronJob run. Use concurrencyPolicy: Forbid (see the deployment document).
PostgreSQL	Managed PostgreSQL 16	System of record, outbox, audit chain	Vertical, plus a read replica. Partitioning at scale (\$10).
SPA	Static files in public/ served by the API	Staff console, customer app, certificate verification	CDN-cacheable /assets/* (Cache-Control: max-age=86400)

5. C4 Level 3 — Components (API server)

Inside the API server, the HTTP adapter calls application services, which use the domain, the rules engine and the driven adapters.



6. Hexagonal layering and the dependency rule



Dependency rule. Source dependencies point inward only:

1. `src/domain/*` and `src/rules/*` depend only on `src/shared/*`. They do no I/O and read no clock (`today` is passed in), so they can be unit-tested deterministically.
2. `src/application/*` depends on the domain, plus **ports** injected through the factory arguments { `store`, `rules`, `audit`, `events`, `clock`, `logger`, `metrics`, `gateways` }. It never requires an adapter.
3. Adapters depend on application/domain types. Only `src/bootstrap/container.js` knows which adapter implements which port. For example, `createPostgresStore` is chosen when `DATABASE_URL` is set, and `createMemoryStore` otherwise.

Ports (implicit interfaces, JS + JSDoc; ADR-002)

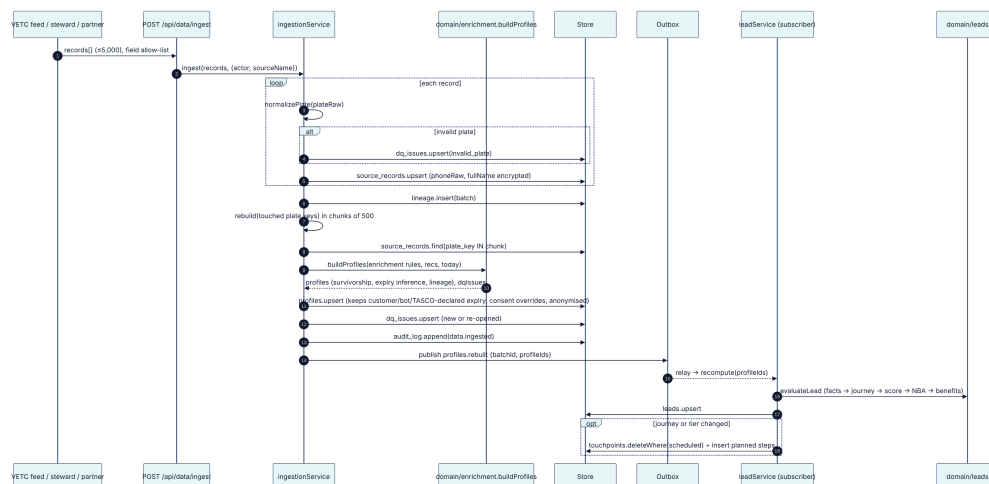
Port	Methods	Adapters
Store / Collection	get, insert, update (optimistic version), upsert, bulkUpsert, find({where, orderBy, limit, offset}), count, countBy, delete, deleteWhere, raw	memoryStore.js, postgresStore.js
Store extras	transaction(fn), claimEvents(limit), migrate(), ping(), close(), audit. {append, list, verify, count}	same
EventBus	publish(type, payload, {actor, correlationId}), subscribe(type, name, fn), relay(limit), drain()	outboxEventBus.js
PaymentGateway	debit({idempotencyKey, customerId, amount, description}), refund({transactionId, amount}). Debits are made only for customer-confirmed orders.	createVetcoWalletGateway (sandbox)
PolicyAdministration	issuePolicy({...}), cancelPolicy({policyNo, reason})	createTascoCoreGateway (sandbox)
NotificationChannel	send({to, text, templateKey, idempotencyKey})	createNotificationGateway({channel}) ×3 (sandbox)
Telephony	runCall(dialogue, session, plateDisplay)	createSimulatedCaller (sandbox)
SourceFeed	fetchBatch()	createSyntheticVetcoSource (sandbox)
RulesProvider	get(kind), getRecord(kind)	rulesService.js

Full contracts are in the [Integration Architecture](#).

7. Key flows

The following sequences show the main runtime paths and the code that implements each step.

7.1 Ingest → golden record → score → journey

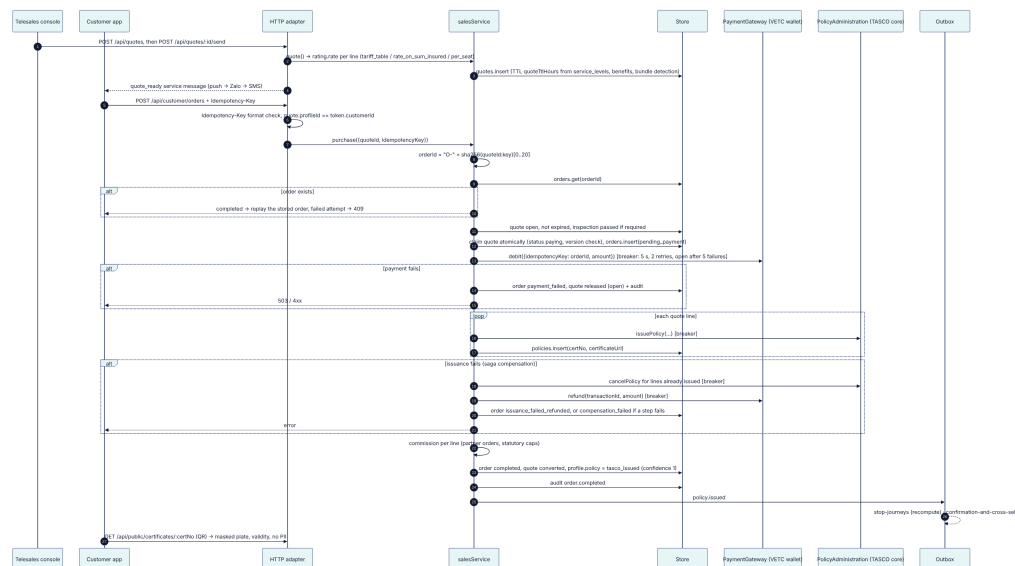


Notes. The rebuild is **incremental by plate**: only the plates in the batch are rebuilt. Facts captured on the platform (`customer_declared`, `voice_bot`, `tasco_issued`) survive a rebuild when they have higher confidence (`ingestionService.rebuild`). A vehicle with an active TASCO TND policy gets NBA `insured`, and its journey is cleared (`LeadService.recomputeOne`).

7.2 Voice bot call → handoff

An automated call runs only after the contact policy allows it, and every outcome updates data or consent.

Staff never take payment. Telesales and supervisors quote (`POST /api/quotes`) and send the quote to the customer's VETC app or Zalo as a service message (`POST /api/quotes/:id/send`). The customer sees open quotes (`GET /api/customer/quotes`), or creates one in the app (`POST /api/customer/quotes`), and pays inside the app. Physical damage cover (`MOTOR_PD`) needs a recorded vehicle inspection (`POST /api/quotes/:id/inspection`) before payment.

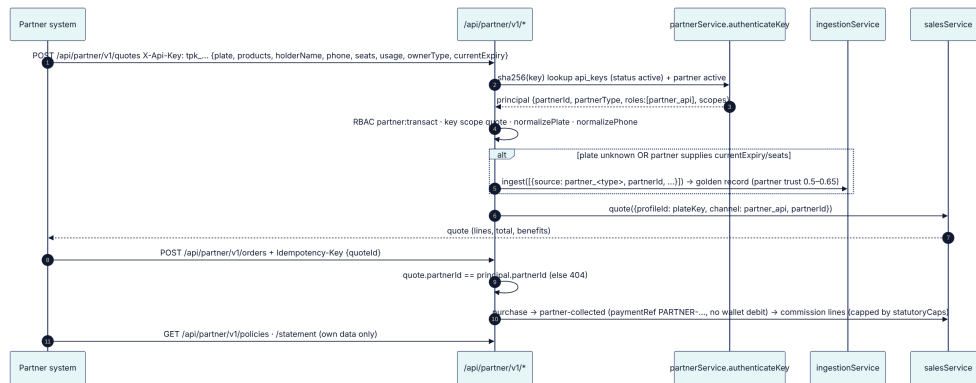


1. **Order id** is derived from (`quoteId`, `Idempotency-Key`). Replaying a completed request returns the stored order; reusing the key of a failed attempt returns HTTP 409, so the client retries with a new key.
2. **Quote claim**. The quote moves from `open` to `paying` with an optimistic version check before any payment, so two concurrent purchases with different keys cannot both debit the wallet; the loser receives 409.
3. The **payment idempotency key** is the order id. The wallet adapter must deduplicate on it; the sandbox does (`processed` map).
4. The **message idempotency key** is the message id.

Follow-up (tracked in the findings register of the [Security Architecture](#), G18): retry failed refunds automatically through a `payment.refund_requested` outbox event with backoff and a dead letter.

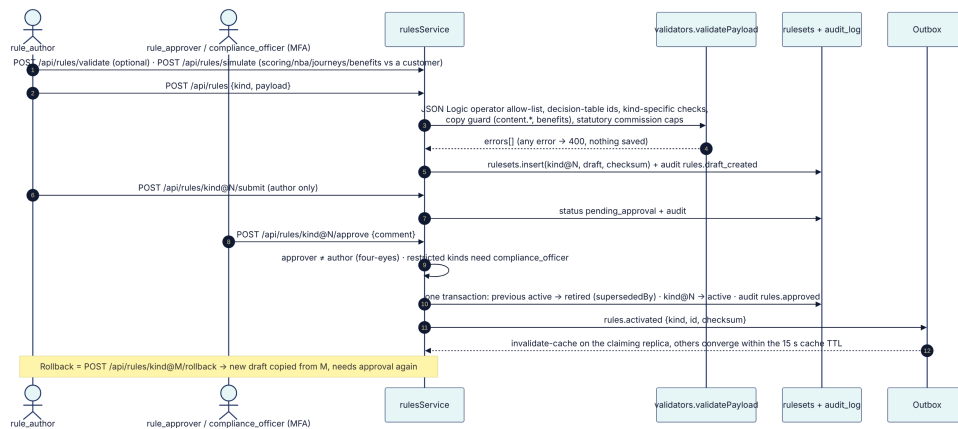
7.4 Partner API new-business onboarding

Partners quote by plate; unknown vehicles are onboarded into the golden record, and the partner collects the premium.

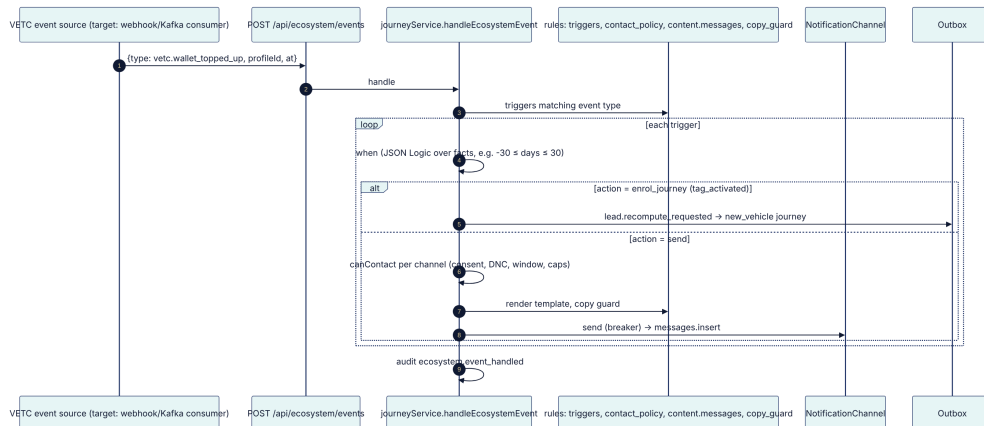


7.5 Rule change with maker-checker

Every rule change is validated, submitted by its author and approved by a different person before it becomes active.



7.6 Ecosystem event trigger (moment of truth)



Today the ecosystem event endpoint is a staff-authenticated API (`perm: journeys:run`). The production source is a VETC event stream; see section 3.6 of the [Integration Architecture](#).

8. Event-driven design

The platform uses domain events for **reactions that must not block the caller**: re-scoring, sending confirmations and acknowledgements, cache invalidation. Later it will also use them for **integration fan-out**.

The current implementation (`src/adapters/messaging/outboxEventBus.js`, [ADR-005](#)) works as follows:

- `publish()` inserts into the `domain_events` table with `status = 'pending'`.
- `relay()` claims a batch with `UPDATE ... WHERE id IN (SELECT ... FOR UPDATE SKIP LOCKED)` (`postgresStore.claimEvents`). Rows left in `processing` for more than 5 minutes (for example after a crash) are reclaimed.

- Each subscriber is a named handler. Handlers that succeed are recorded in `evt.handled`, so on a retry only the failed handlers run again.
- After 5 attempts the event moves to `dead_letter`.
- Delivery is **at-least-once**, so handlers must be idempotent.
- The relay runs every 1 s in each API replica (`src/server.js`), as the `relay CronJob`, and synchronously via `events.drain()` at the end of interactive requests that need read-your-writes (for example `POST /api/customer/orders`).

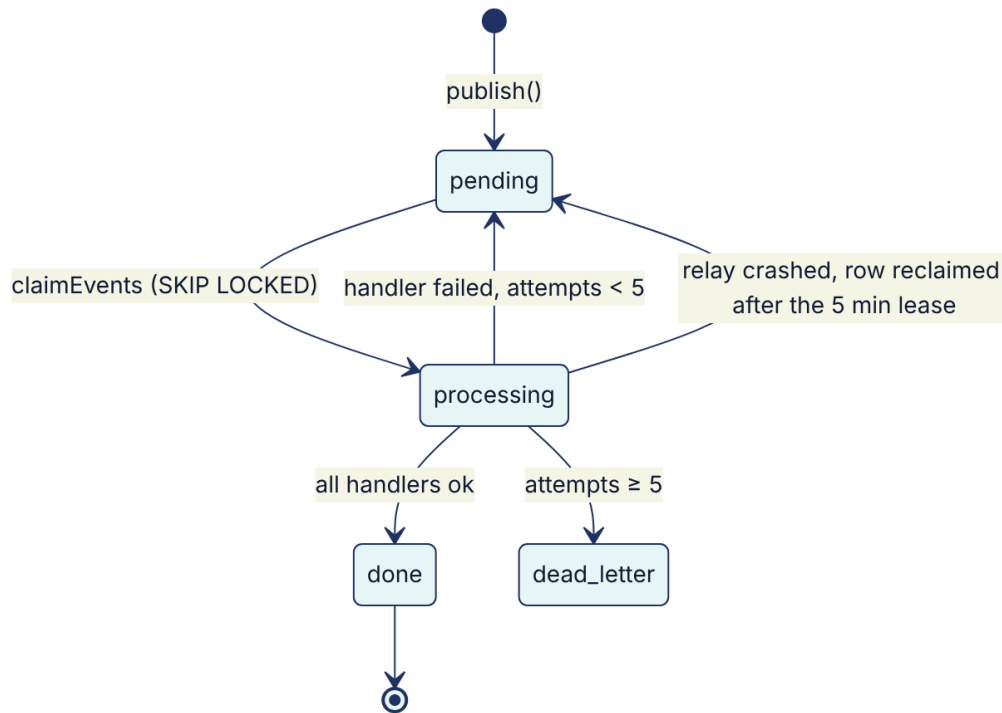
8.1 Event catalogue

Event	Producer (file)	Payload	Subscribers (<code>src/relay/relay.js</code>)	Idempotency of handler
<code>profiles.rebuilt</code>	<code>ingestionService.ingest</code>	<code>{batchId, profileIds[]}</code>	<code>recompute-leads → leadService.recompute(ids)</code>	Recompute is a pure function of current state; upserts by profile id
<code>lead.recompute_requested</code>	<code>journeyService.handleEcosystemEvent (enrol_journey), voiceService.finalize (opt-out, already-renewed), customerService.declareExpiry/updateConsent</code>	<code>{profileIds[], reason}</code>	<code>recompute-leads</code>	Same as above
<code>policy.issued</code>	<code>salesService.purchase</code>	<code>{orderId, profileId, policies[{id, product, endDate}], channel, journey}</code>	<code>stop-journeys (recompute → NBA insured, deletes scheduled touchpoints), confirmation-and-cross-sell (service message on push/Zalo; cross-sell touchpoints with deterministic ids if marketing consent)</code>	Touchpoint ids are deterministic (<code>profile:cross_sell:step:due</code>). The confirmation message is not deduplicated on replay (new message id each time); see ADR-005, gap 6.
<code>renewal.link_requested</code>	<code>voiceService.finalize (link_sent)</code>	<code>{profileId, journey}</code>	<code>send-link (push → Zalo → SMS, non-marketing, first sent wins)</code>	Not deduplicated on replay (same caveat)
<code>handoff.created</code>	<code>voiceService.finalize (hot_handoff)</code>	<code>{handoffId, profileId}</code>	<code>none today. Target: CTI / dialler queue, supervisor alert</code>	n/a
<code>claim.submitted</code>	<code>claimsService.submit</code>	<code>{claimId, profileId}</code>	<code>acknowledge-claim (claim_received service message, push → Zalo → SMS, with the SLA <code>claimAckSLAHours</code> from <code>service_levels</code>). Target: TASCO core claims FNOL API.</code>	Not deduplicated on replay (same caveat)
<code>rules.activated</code>	<code>rulesService.approve</code>	<code>{kind, id, checksum}</code>	<code>invalidate-cache</code>	Naturally idempotent

Events carry `id`, `type`, `actor`, `correlationId` (nullable today; target: the HTTP `X-Request-Id`), `occurredAt`, `status`, `attempts`, `handled[]`, `lastError`, `processedAt`.

8.2 Event lifecycle

Each event moves through the following states.



9. Configurability model

Business behaviour lives in rule sets. Each rule set is versioned in `rulesets (kind@N)`, validated per kind (`src/rules/validators.js`) and activated only by a second person. Code changes are needed only for a new **rating method**, **dialogue state**, **rule operator** or **integration**.

Rule kind	File	What it controls	Business owner	Change path
products	products.json	Catalogue, channels, rating method + rate kind, bundles	Product	Rules console, maker-checker
tariff.tnds_car, tariff.tnds_motorbike	tariff.*.json	Regulated TNDs premiums (Decree 67/2023, confirm with TASCO legal), VAT	Underwriting + Compliance	Maker-checker (compliance approval recommended)
rating.motor_pd, rating.pa_seat	rating.*.json	Illustrative voluntary rates. Replace with TASCO filed rates.	Actuarial	Maker-checker
commission	commission.json	Partner commission table + statutoryCaps. The validator rejects rates above the cap.	Finance + Legal	Maker-checker; approver must be <code>compliance_officer</code>
enrichment	enrichment.json	Source trust, expiry evidence weights, corroboration, category table, DQ weights	Data office	Maker-checker + DQ regression check
scoring	scoring.json	Factors (weights must sum to 100), tiers, damping, zeroWhen	Growth analytics	Maker-checker + simulate + model-risk review (AI and Prompt Governance)
nba	nba.json	Next-best-action decision table	Campaign	Maker-checker + simulate
journeys	journeys.json	Audiences, anchors, steps, channels, marketing flag, cross-sell	Campaign	Maker-checker + simulate
triggers	triggers.json	Ecosystem event → action	Campaign	Maker-checker
benefits	benefits.json	Value-beyond-discount catalogue. Only <code>legalStatus=approved</code> and available items reach customers; loyalty points stay pending legal review.	Product + Legal	Maker-checker, copy guard
content.messages	content.messages.json	Message templates (vi/en)	Marketing + Compliance	Maker-checker, copy guard. Zalo template registration is out of band (Integration Architecture, section 4).
content.voicebot	content.voicebot.json	Bot script lines and NLU keywords	CX + Compliance	Maker-checker, copy guard (AI and Prompt Governance)
copy_guard	copy_guard.json	Banned phrases (no discounts or rebates on regulated products), including whitespace and punctuation evasions	Compliance	Maker-checker; approver must be <code>compliance_officer</code>
contact_policy	contact_policy.json	Contact window, frequency caps, consent per channel	Compliance / DPO	Maker-checker; approver must be <code>compliance_officer</code>

Rule kind	File	What it controls	Business owner	Change path
abac	abac.json	Attribute policies (region, own handoffs)	InfoSec	Maker-checker; approver must be compliance_officer (InfoSec consulted)
retention	retention.json	Retention days and action per entity	DPO + Legal	Maker-checker; approver must be compliance_officer
service_levels	service_levels.json	Quote TTL (24 h), claim acknowledgement SLA (4 h), evidence confidences for customer, steward and voice-bot facts	Operations + Data office	Maker-checker
referral	referral.json	Referral programme (disabled pending legal)	Product + Legal	Maker-checker
costs	costs.json	Unit costs for the business-case dashboard	Finance	Maker-checker
RBAC	config/security/rbac.json	Role → permission	InfoSec	Code review + CAB (not a rule kind, deliberately)

Guard-rails present in code:

- the JSON Logic operator allow-list and forbidden keys (`__proto__`, `prototype`, `constructor`);
- weights must sum to 100;
- `tiers.hot > tiers.warm`;
- unique ids;
- a valid contact window;
- VAT between 0 and 0.2;
- the copy guard on every customer-facing string;
- commission caps;
- one active version per kind (unique index, migration `002_integrity_hardening.sql`), with activation in a single transaction.

Approval writes `rules.activated`. The rules cache TTL is 15 s (`CACHE_TTL_MS`).

10. Scalability to 6 million vehicles

Assumptions:

- 6.0 M vehicles;
- about 1.3 source records per vehicle (7.8 M rows);
- expiries spread evenly, so about 16.4 k vehicles reach expiry per day;
- about 30 % of the base in an active journey at any time;
- detailed sizing in section 12 of the [Deployment and Infrastructure Architecture](#).

Concern	Design already in code	Required at 6 M (planned)
Golden-record build	Incremental rebuild by plate in 500-plate chunks (<code>ingestionService.rebuild</code>). Only touched plates are rebuilt.	Initial load: bulk COPY into a staging table, then rebuild in parallel workers partitioned by <code>hash(plate_key) % N</code> (each worker a Job). Steady state: VETC CDC → small batches.
Lead recompute	Event-driven for changed profiles	The full recompute (<code>recompute(null)</code>) uses <code>OFFSET</code> paging, which degrades at millions of rows. Move to keyset pagination (<code>WHERE id > \$last ORDER BY id</code>) and shard across workers. Run nightly only for time-dependent facts (days).
Journey execution	<code>runDue</code> pages through all due touchpoints in batches of 1,000; touchpoints blocked only by the contact window are deferred, not skipped	Claim touchpoints with <code>FOR UPDATE SKIP LOCKED</code> (as the outbox does) so N workers can run in parallel without double sends. Today safety relies on <code>CronJob concurrencyPolicy: Forbid</code> .
Table growth	Indexed columns per collection	Range-partition messages and touchpoints by month, and <code>domain_events</code> by <code>occurred_at</code> with a purge of done events older than 30 days. Hash-partition <code>profiles/leads/source_records</code> by <code>id</code> only if the table exceeds about 100 GB.
Reads for BI and dashboards	<code>countBy</code> on indexed columns	Route <code>/api/dashboard/*</code> and the analytics feed to a read replica . Replace <code>insightsService.governance()</code> full-chain <code>audit.verify()</code> with incremental verification from a checkpoint (ADR-008).
Outbound volume	Breakers per channel	Queue offload: outbox → broker topic per channel → channel workers with provider rate limits (Zalo ZNS quotas, SMS TPS). See ADR-005 .

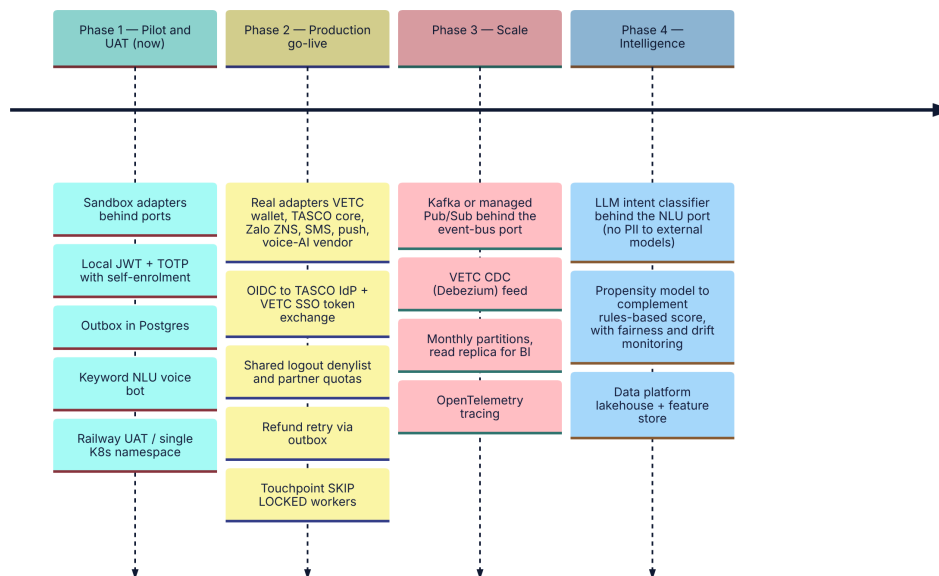
Concern	Design already in code	Required at 6 M (planned)
Audit throughput	<code>pg_advisory_xact_lock(724001)</code> serialises appends to keep one linear chain	About 1–2 k appends/s on modest hardware is enough for the current audit points. If needed, use per-day chains or batched Merkle roots.
API	Stateless; HPA; edge rate limit at the ingress	Shared logout denylist and per-partner quotas (Redis or gateway) once there is more than one replica (Security Architecture, finding G7).

11. Quality attributes (targets)

Attribute	Target	Mechanism
Availability (API)	99.9 % monthly	≥ 2 replicas across zones, PDB, readiness gating on DB ping, graceful drain (<code>src/server.js</code>)
Latency	p95 < 300 ms for read APIs; p95 < 1.5 s for purchase (dominated by wallet and core)	Indexed column filters; <code>statement_timeout 15 s</code> ; breaker timeouts
RPO / RTO	RPO ≤ 5 min (PITR), RTO ≤ 1 h (prod)	Managed Postgres PITR + cross-region replica in DR
Rule change lead time	≤ 1 day from draft to active	Maker-checker console; target published in <code>insightsService.adoption()</code>
Explainability	100 % of scores carry factor reasons; 100 % of NBAs carry <code>ruleId</code>	<code>domain/leads.js</code>
Auditability	Every state-changing use case audited; chain verifiable	<code>auditService</code> , <code>/api/audit/verify</code>

12. Evolution roadmap

Platform evolution (indicative)



Item	Why	Approach	Constraint kept
Kafka / Pub/Sub	Fan-out to CTI, data platform, TASCO core; replay	Relay publishes outbox rows to topics; consumers keep the named-handler idempotency model	Publishers do not change (<code>events.publish</code>)
LLM NLU	Better intent recall on free speech	Implement <code>classify(text) → intent</code> with the same contract as <code>keywordClassifier</code> , behind a feature flag; evaluate against a labelled set	The dialogue state machine and script stay deterministic and governed (ADR-009)
Real IdP	Joiner-mover-leaver, SSO, central MFA	OIDC code flow with PKCE (staff); RS256 tokens verified by JWKS; roles from IdP groups	Principal shape <code>{id, roles, region}</code> unchanged (ADR-007)
Data platform	BI, model training, regulatory reporting	Logical replication / CDC of the indexed columns into pseudonymised curated marts (no decrypted PII)	PII is minimised and pseudonymised by plate-key HMAC (Data Architecture, section 11)